BURP SUITE: REPEATER

TryHackMe Learning & Practical Report

Prepared by	Belal Rabea Khalifa
Program	IqraSec Academy, WAPT Internship (Cohort 02), Phase 1
Task	3. Burp Suite
Platform	TryHackMe
Room	Burp Suite: Repeater (https://tryhackme.com/room/burpsuiterepeater)
Focus	Burp Repeater, Inspector, request modification, Union SQL injection
Completion	100% (all 9 tasks)
Date	October 2026

1. Objective

Learn how to use Burp Suite Repeater to edit and resend HTTP requests, read the responses, and test how an application behaves, including a Union-based SQL injection in the room's challenge.

2. Concept / Understanding

- **Repeater:** a Burp tool to take a request (for example from the Proxy), change it by hand, and send it again as many times as needed, then compare the responses.
- **Proxy to Repeater:** capture the request with Intercept on, then send it to Repeater (Ctrl+R).
- **Message views:** Pretty (formatted), Raw (exact message), Hex (bytes), Render (how a browser would show the page).
- **Inspector:** a side panel that splits a request into parts (attributes, query parameters, body parameters, cookies, headers) so they are easy to read and edit.
- **SQL injection (Union-based):** when user input goes straight into an SQL query, an attacker can add *UNION SELECT* to read data from other tables. The number of columns must match the original query, so *null* is used as filler. A single quote (') is a quick test: if it breaks the query, a database error may appear.
- **information_schema:** a database that lists the tables and columns, used to find what to extract.

3. Practical Work

I finished all 9 tasks of the room. These were the practical steps:

#	Task	What I did
1	Basic usage	Captured a request with the Proxy and sent it to Repeater (Ctrl+R). Explored the Request and Response panels, the message views and the Inspector.
2	Inspector	Added the header `FlagAuthorised: True` to the request and sent it. The response gave the flag.
3	Challenge (Task 7)	Captured a request to a numeric products endpoint and sent it to Repeater. Changed the number at the end to an extreme value and got **500 Internal Server Error**, which gave the flag.
4	Extra-mile (Task 8)	On the /about/ID endpoint, a single quote (') caused a database error, so the input is used in an SQL query. Then I used `UNION ALL SELECT` to read the notes of the person with id 1 from the *people* table and got the flag.

4. Tools Used

- Burp Suite (Proxy, Repeater, Inspector)
- Web browser
- TryHackMe AttackBox and the room's target machine

5. Evidence

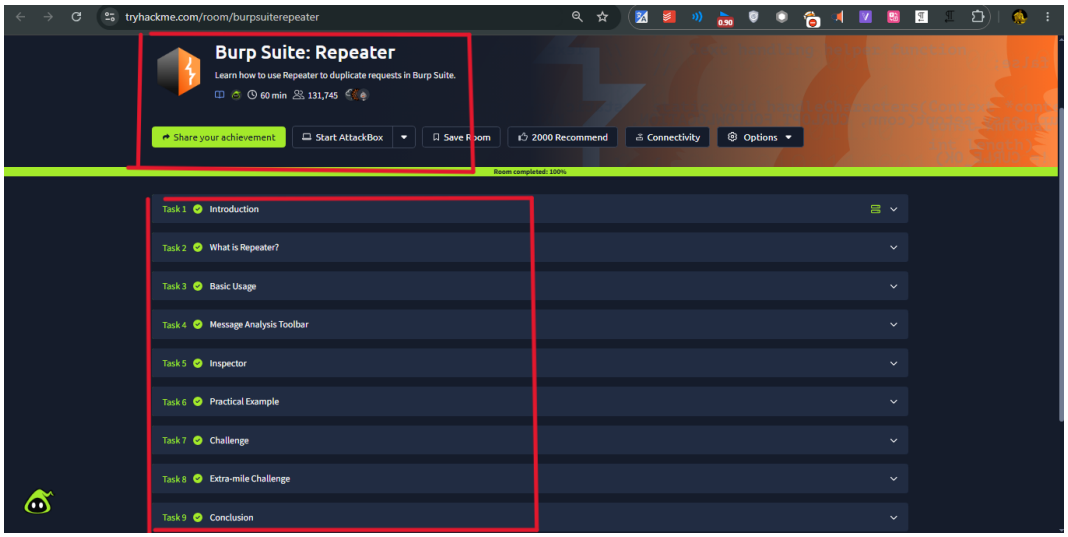


Figure 1: Burp Suite: Repeater room page, 100% completed, all 9 tasks done.

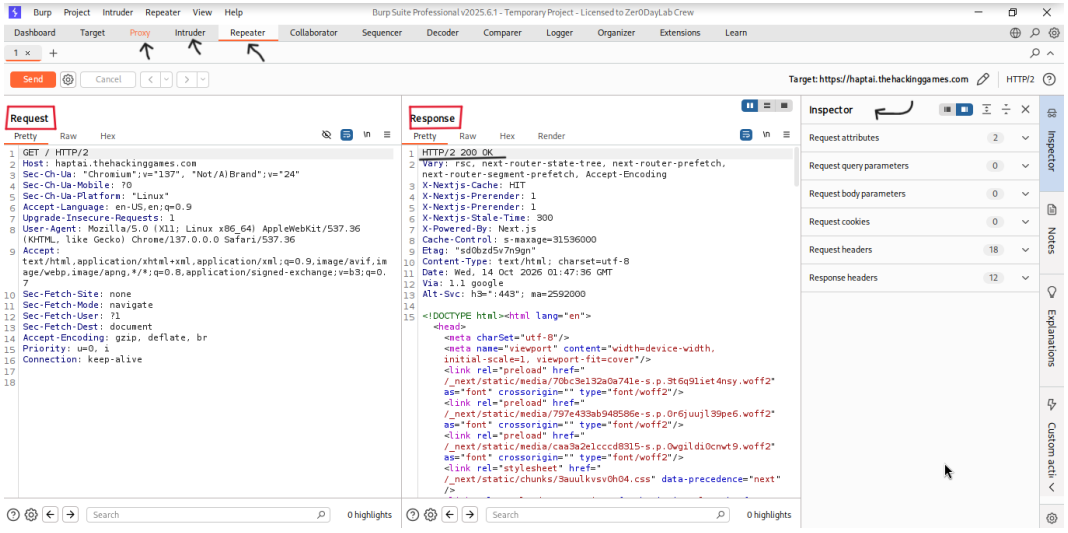


Figure 2: Burp Repeater interface: Request panel (left), Response panel (right), Inspector (far right). My AttackBox time for the day was finished, so I took this screenshot with a simple GET request to the home page of a public website, only to show the interface. I completed the room earlier (see Figure 1 and the accepted answers in Figures 4 to 6).

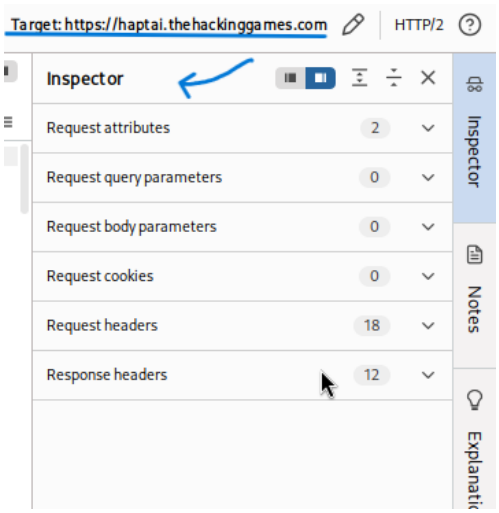


Figure 3: the Inspector panel (same example request), showing the parts of the request: attributes, parameters, cookies, headers.

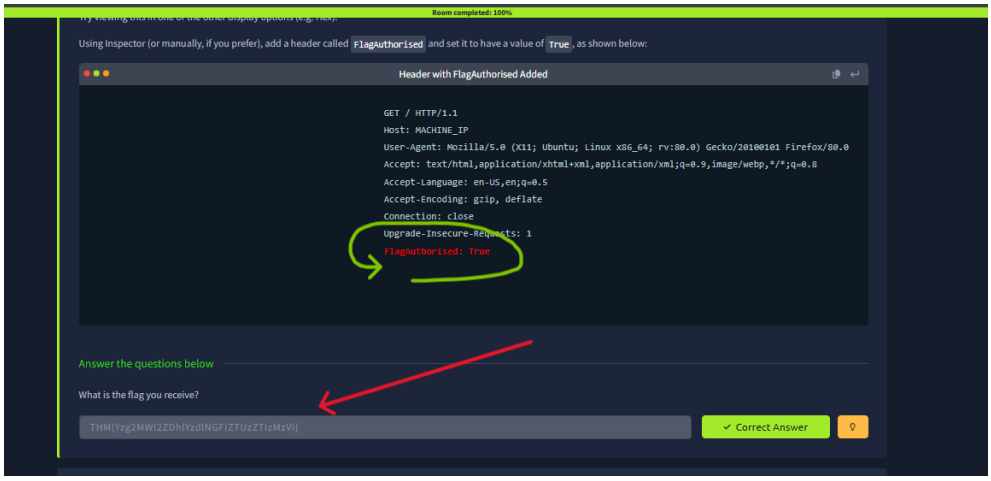


Figure 4: the header FlagAuthorised: True added to the request, and the flag accepted as correct.

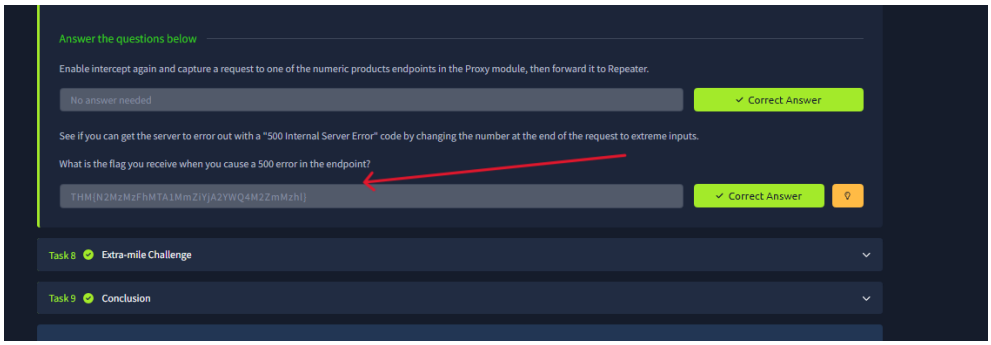


Figure 5: Task 7, the flag for the 500 Internal Server Error accepted as correct.

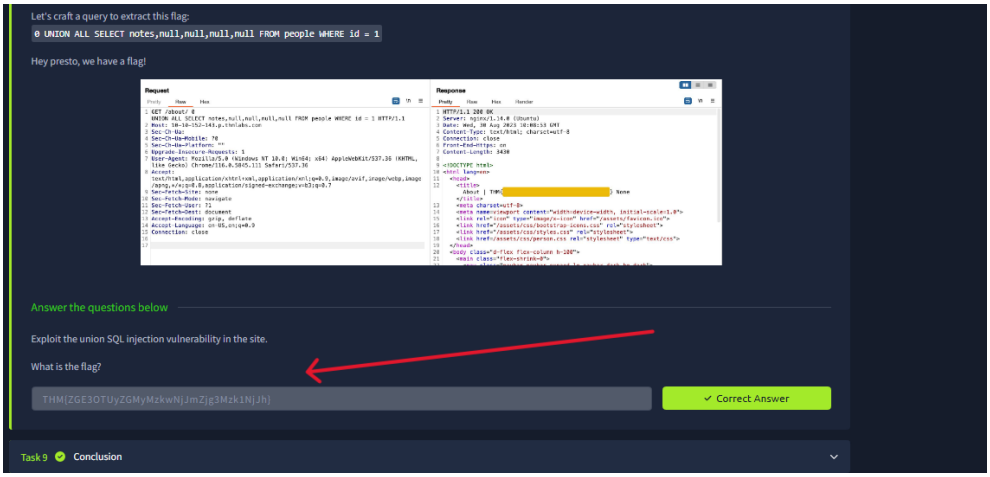


Figure 6: Task 8, the Union SQL injection flag accepted as correct.

6. Result

Completed the room 100%. I used Repeater to modify and resend requests, added a custom header, caused a 500 error with an extreme input, and exploited the Union-based SQL injection to get the flags. All answers were accepted.

7. Key Learnings

- Repeater lets me change one thing in a request and see exactly how the server reacts.
- The Inspector makes it easy to edit headers, parameters and cookies.
- Extreme or unexpected input can show how the server handles errors.
- A single quote is a quick test for SQL injection, and UNION SELECT can read data from other tables.

8. Impact and Remediation (SQL Injection)

Impact: an attacker can read data from the database directly, like other tables, columns and sensitive records (users, notes, credentials), without being logged in.

Remediation:

- Use parameterized queries (prepared statements), so user input is never run as SQL code.
- Validate and limit the input (for example, only accept numbers for an ID).
- Do not show detailed database errors to users in production.